

Análisis de Metadatos II: Unidad 2- Parte 3

Lenguaje de programación Python

Jair Pallares Castellon, MSc.



Manipulación de estructuras: Listas

Manipulación de estructuras

Listas

Cuando hablamos de secuencias de números enteros o flotantes o incluso de cadenas, hablamos entonces de listas. En Python los valores de una lista deben estar encerrados entre corchetes y separados por comas.

```
>>> [1, 2, 3]
```

```
[1, 2, 3]
```

```
>>> a = [1, 2, 3]
```

```
>>> a
```

```
[1, 2, 3]
```

```
>>> nombres = ['Juan', 'Antonia', 'Luis', 'Maria']
```

```
>>> a = [ ]
```

```
>>> print(a)
```

```
[ ]
```

Manipulación de estructuras

cosas que ya sabemos sobre las listas y hemos usado sin darnos cuenta

Muchos de los operadores que se usan en las cadenas, también pueden ser usados en las listas.

```
>>> a = [1, 2, 3]
```

```
>>> len(a)
```

```
3
```

```
>>> len([])
```

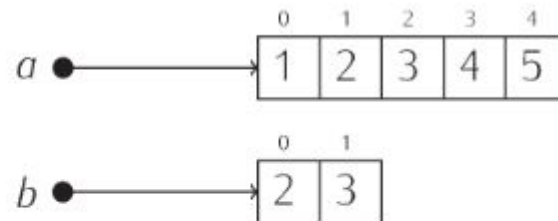
```
1
```

```
>>> [1, 2] + [3, 4]
```

```
[1, 2, 3, 4]
```

```
>>> a[2]
```

```
3
```



```
>>> a = [1, 2, 3, 4, 5] ↵
```

```
>>> b = a[1:3] ↵
```

```
>>> c = a ↵
```

Manipulación de estructuras

cosas que ya sabemos sobre las listas y hemos usado sin darnos cuentas

El iterador **for-in** también recorre los elementos de una lista:

```
>>> for i in [1, 2, 3]:  
...     print(i)  
...  
1  
2  
3
```

```
>>> for i in range(1, 4):  
...     print(i)  
...  
1  
2  
3
```

```
>>> a = list(range(1, 4))  
>>> print(a)  
[1, 2, 3]
```

```
>>> [0] * 10  
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Comparación de listas

Los operadores de comparación también trabajan con listas. Parece claro cómo se comportarán operadores como los de igualdad (==) o el de desigualdad (!=):

- Si las listas son de talla diferente, resolviendo que las listas son diferentes.
- y si miden lo mismo, comparando elemento a elemento de izquierda a derecha y resolviendo que las dos listas son iguales si todos sus elementos son iguales, y diferentes si hay algún elemento distinto.

Los otros operadores >, <, >=, <= también funcionan con las listas.

```
>>> [1, 2, 3] == [1, 2]↵  
False  
>>> [1, 2, 3] == [1, 2, 3]↵  
True  
>>> [1, 2, 3] == [1, 2, 4]↵  
False
```

Manipulación de estructuras

Operador especial is

El operador **is** devuelve **True** si dos objetos son en realidad el mismo objeto, es decir, si residen ambos en la misma zona de memoria, y **False** en caso contrario.

```
>>> a = [1, 2, 3]↵  
>>> b = [1, 2, 3]↵  
>>> c = a↵  
>>> a is b↵  
False  
>>> a is c↵  
True
```

```
>>> a = [1, 2]↵  
>>> a is [1, 2]↵  
False  
>>> a == [1, 2]↵  
True
```

← Son equivalentes elemento
a elemento

Manipulación de estructuras

Modificación de elementos de listas

Sabemos cómo crear una lista y a consultar su contenido. Aquí aprenderemos a como modificar su contenido.

Cada celda de una lista es, en cierto modo, una variable autónoma: podemos almacenar en ella un valor y modificarlo a voluntad.

```
>>> a = [1, 2, 3]↵  
>>> a[1] = 10↵  
>>> a↵  
[1, 10, 3]
```

Adición de elementos de una lista

El operador siempre debe trabajar con dos listas o usando el método **append()**

```
>>> a = [1, 2, 3]↵  
>>> a = a + [4]↵  
>>> a↵  
[1, 2, 3, 4]
```

```
>>> a = [1, 2, 3]↵  
>>> a.append(4)↵  
>>> a↵  
[1, 2, 3, 4]
```


Manipulación de estructuras

Ejemplo: Vamos a construir un programa que construya una lista con todos los números primos entre 1 y n. Como no sabemos a priori cuántos hay, construiremos una lista vacía e iremos añadiendo números primos conforme los vayamos encontrando.

```

obten_primos.py - C:/Users/angoq/AppData/L...
File Edit Format Run Options Window Help
1 n = int(input('Introduce el valor máximo: '))
2
3 primos = []
4 for número in range(2, n + 1):
5     # Determinamos si el número es primo.
6     creo_que_es_primo = True
7     for divisor in range(2, número):
8         if número % divisor == 0:
9             creo_que_es_primo = False
10            break
11        # Y si es primo, lo añadimos a la lista.
12        if creo_que_es_primo:
13            primos.append(número)
14
15 print(primos)
Ln: 15 Col: 12

```

```

IDLE Shell 3.10.3
File Edit Shell Debug Options Window Help
Python 3.10.3 (tags/v3.10.3:a342a49, Mar 16
2022, 13:07:40) [MSC v.1929 64 bit (AMD64)]
on win32
Type "help", "copyright", "credits" or "lice
nse()" for more information.
>>>
= RESTART: C:/Users/angoq/AppData/Local/Prog
rams/Python/Python310/obten_primos.py
Introduce el valor máximo: 4
[3]
>>>
= RESTART: C:/Users/angoq/AppData/Local/Prog
rams/Python/Python310/obten_primos.py
Introduce el valor máximo: 6
Ln: 11 Col: 0

```